

Understanding the Anatomy of an Android Application

Mastering Important Android Terminology

This chapter introduces you to the terminology used in Android application development and provides you with a more thorough understanding of how Android applications function and interact with one another. Here are some of the important terms covered in this chapter:

Context: The context is the central command center for an Android application. Most application-specific functionality can be accessed or referenced through the context. The Context class (`android.content.Context`) is a fundamental building block of any Android application and provides access to application-wide features such as the application's private files and device resources, as well as system-wide services. The application-wide Context object is instantiated as an Application object (`android.app.Application`).

Activity: An Android application is a collection of tasks, each of which is called an activity. Each activity within an application has a unique task or purpose. The Activity class (`android.app.Activity`) is a fundamental building block of any Android application, and most applications are made up of several activities. Typically, the purpose is to handle the display of a single screen, but thinking only in terms of "an activity is a screen" is too simplistic. An Activity class extends the Context class, so it also has all of the functionality of the Context class.

Intent: The Android operating system uses an asynchronous messaging mechanism to match task requests with the appropriate activity. Each request is packaged as an intent. You can think of each such request as a message stating an intent to do something. Using the Intent class (`android.content.Intent`) is the primary method by which application components such as activities and services communicate with one another.

Service: Tasks that do not require user interaction can be encapsulated in a service. A service is most useful when the operations are lengthy (offloading time-consuming processing) or need to be done regularly (such as checking a server for new mail). Whereas activities run in the foreground and generally have a user interface, the Service class (`android.app.Service`) is used to handle background operations related to an Android application. The Service class extends the Context class.

The Application Context

The application Context is the central location for all top-level application functionality. The Context class can be used to manage application-specific configuration details as well as application-wide operations and data. Use the application Context to access settings and resources shared across multiple Activity instances.

Retrieving the Application Context

You can retrieve the Context for the current process using the `getApplicationContext()` method, found in common classes such as Activity and Service, like this:

```
Context context = getApplicationContext();
```

Using the Application Context

After you have retrieved a valid application Context object, it can be used to access application-wide features and services, including the following:

Retrieving application resources such as strings, graphics, and XML files

Accessing application preferences

Managing private application files and directories

Retrieving uncompiled application assets

Accessing system services

Managing a private application database (SQLite)

Performing Application Tasks with Activities

The Android Activity class (`android.app.Activity`) is the core of any Android application. Much of the time, you define and implement an Activity class for each screen in your application. For example, a simple game application might have the following five activities, as shown in Figure 4.1.

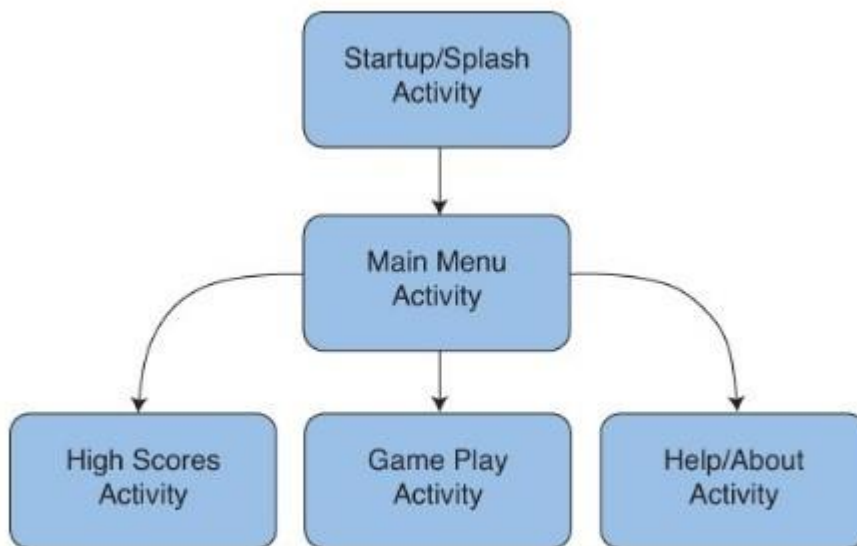


Figure 4.1 A simple game with five activities.

Figure 4.1 A simple game with five activities.

A startup or splash screen: This activity serves as the primary entry point to the application. It displays the application name and version information, and transitions to the main menu after a short interval.

A main menu screen: This activity acts as a switch to drive users to the core activities of the application. Here, users must choose what they want to do within the application.

A game play screen: This activity is where the core game play occurs.

A high scores screen: This activity might display game scores or settings.

A help/about screen: This activity might display the information users might need to play the game.

The Lifecycle of an Android Activity

Android applications can be multiprocess, and the Android operating system allows multiple applications to run concurrently, provided memory and processing power are available. Applications can have background behavior, and applications can be interrupted and paused when events such as phone calls occur. Only one active application can be visible to the user at a time—specifically, a single application Activity is in the foreground at any given time.

The Android operating system keeps track of all Activity objects running by placing them on an Activity stack (see Figure 4.2). This Activity stack is referred to as the “back stack.” When a new Activity starts, the Activity on the top of the stack (the current foreground Activity) pauses and the new Activity pushes onto the top of the stack. When that Activity finishes, it is removed from the Activity stack and the previous Activity in the stack resumes.



Figure 4.2 The Activity stack.

Android applications are responsible for managing their state as well as their memory, resources, and data. They must pause and resume seamlessly. Understanding the different states within the Activity lifecycle is the first step in designing and developing robust Android applications.

Using Activity Callbacks to Manage Application State and Resources

Different important state changes within the Activity lifecycle are punctuated by a series of important method callbacks. These callbacks are shown in Figure 4.3.

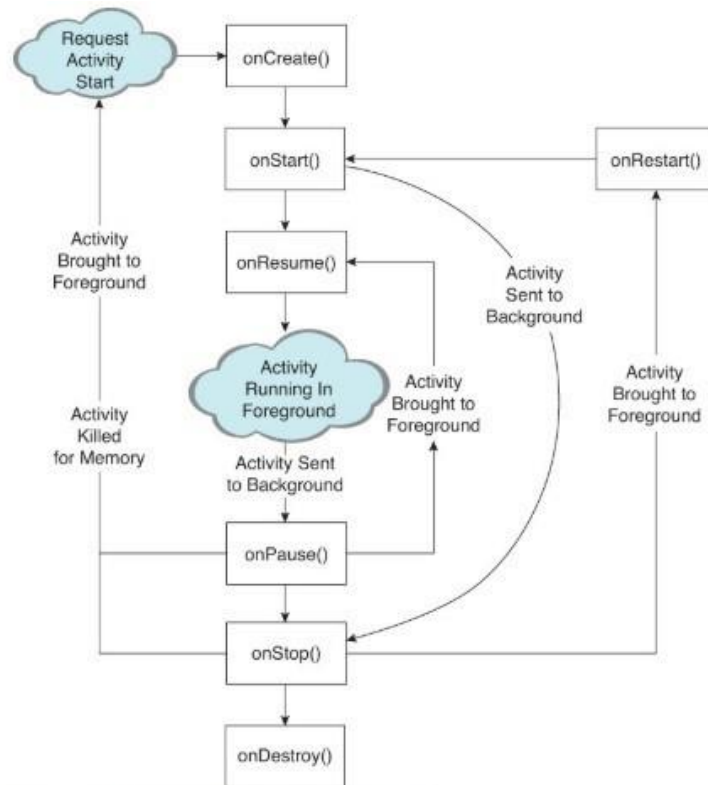


Figure 4.3 The lifecycle of an Android Activity.

Figure 4.3 The lifecycle of an Android Activity.

Here are the method stubs for the most important callbacks of the Activity class

```

public class MyActivity extends Activity {
    protected void onCreate(Bundle savedInstanceState);
    protected void onStart();
    protected void onRestart();
    protected void onResume();
    protected void onPause();
    protected void onStop();
    protected void onDestroy();
}

```

Now let's look at each of these callback methods, when they are called, and what they are used for.

Initializing Static Activity Data in onCreate()

When an Activity first starts, the onCreate() method is called. The onCreate() method has a single parameter, a Bundle, which is null if this is a newly started Activity. If this Activity was killed for memory reasons and is now restarted, the Bundle contains the previous state information for this Activity so that it can reinitiate. It is appropriate to perform any setup, such as layout and data binding, in the onCreate() method. This includes calls to the setContentView() method.

Confirming Features in onStart()

When confirming features for the first time, after the call to onCreate(), or reconfirming after a call to onStop(), then onRestart(), the onStart() method is where you should confirm that the appropriate features are enabled on a user's device. For example, if Bluetooth is required for your application to function properly, the onStart() method is where you would check to verify that Bluetooth has been enabled, and if not, you would request that the user enable Bluetooth before proceeding with your application.

Initializing and Retrieving Activity Data in onResume()

When the Activity reaches the top of the Activity stack and becomes the foreground process, the onResume() method is called. Although the Activity might not be visible yet to the user, this is the most appropriate place to retrieve any instances of resources (exclusive or otherwise) that the Activity needs to run. Often, these resources are the most process intensive, so we keep them around only while the Activity is in the foreground.

TIP

The onResume() method is often the appropriate place to start audio, video, and animations.

Stopping, Saving, and Releasing Activity Data in onPause()

When another Activity moves to the top of the Activity stack, the current Activity is informed that it is being pushed down the Activity stack by way of the onPause() method.

Here, the Activity should stop any audio, video, and animations it started in the onResume() method. This is also where you must deactivate resources such as database Cursor objects or other objects that should be cleaned up should your Activity be terminated. The onPause() method may be the last chance for the Activity to clean up and release any resources it does not need while in the background. You need to save any uncommitted data here, in case your application does not resume. The system reserves the right to kill an Activity without further notice after the call on onPause().

The Activity can also save state information to Activity-specific preferences or application-wide preferences. We talk more about preferences in Chapter 14, "Using Android Preferences."

The Activity needs to perform anything in the onPause() method in a timely fashion because the new foreground Activity is not started until the onPause() method returns.

WARNING

Generally speaking, any resources and data retrieved in the `onResume()` method should be released in the `onPause()` method. If they aren't, there is a chance that these resources can't be cleanly released if the process is terminated.

Avoiding Activities Being Killed

Under low-memory conditions, the Android operating system can kill the process for any Activity that has been paused, stopped, or destroyed. This essentially means that any Activity not in the foreground is subject to a possible shutdown.

If the Activity is killed after `onPause()`, the `onStop()` and `onDestroy()` methods will not be called. The more resources released by an Activity in the `onPause()` method, the less likely the Activity is to be killed while in the background without further state methods being called.

The act of killing an Activity does not remove it from the Activity stack. Instead, the Activity state is saved to a Bundle object, assuming the Activity implements and uses `onSaveInstanceState()` for custom data, although some View data is automatically saved. When the user returns to the Activity later, the `onCreate()` method is called again, this time with a valid Bundle object as the parameter.

TIP

So why does it matter if your application is killed when it is straightforward to resume? Well, it's primarily about responsiveness. The application designer must maintain the data and resources the application needs to resume quickly without degrading the CPU and system resources while paused in the background.

Saving Activity State into a Bundle with `onSaveInstanceState()`

If an Activity is vulnerable to being killed by the Android operating system due to low memory, or in response to state changes like a keyboard opening, the Activity can save state information to a Bundle object using the `onSaveInstanceState()` callback method. This call is not guaranteed under all circumstances, so use the `onPause()` method for essential data commits. What we recommend doing is saving important data to persistent storage in `onPause()`, but using `onSaveInstanceState()` to start any data that can be used to rapidly restore the current screen to the state it was in (as the name of the method might imply).

TIP

You might want to use the `onSaveInstanceState()` method to store nonessential information such as uncommitted form-field data or any other state information that might make the user's experience with your application less cumbersome.

When this Activity is returned to later, this Bundle is passed in to the `onCreate()` method, allowing the Activity to return to the exact state it was in when the Activity paused. You can also read Bundle information after the `onStart()` callback method using the `onRestoreInstanceState()` callback. Thus, when the Bundle information is there, restoring the previous state will be faster and more efficient than starting from scratch.

Destroying Static Activity Data in onDestroy()

When an Activity is being destroyed in the normal course of operation, the `onDestroy()` method is called. The `onDestroy()` method is called for one of two reasons: the Activity has completed its lifecycle voluntarily, or the Activity is being killed by the Android operating system because it needs the resources but still has the time to gracefully destroy the Activity (as opposed to terminating it without calling the `onDestroy()` method).

TIP

The `isFinishing()` method returns `false` if the Activity has been killed by the Android operating system. This method can also be helpful in the `onPause()` method to know if the Activity is not going to resume right away. However, the Activity might still be killed in the `onStop()` method at a later time, regardless. You may be able to use this as a hint as to how much instance-state information to save or permanently persist.

Managing Activity Transitions with Intents

Throughout the lifetime of an Android application, the user might transition between a number of different Activity instances. At times, there might be multiple Activity instances on the Activity stack. Developers need to pay attention to the lifecycle of each Activity during these transitions.

Some Activity instances—such as the application splash/startup screen—are shown and then permanently discarded when the main menu screen Activity takes over. The user cannot return to the splash screen Activity without relaunching the application. In this case, use the `startActivity()` and appropriate `finish()` methods.

Other Activity transitions are temporary, such as a child Activity displaying a dialog and then returning to the original Activity (which was paused on the Activity stack and now resumes). In this case, the parent Activity launches the child Activity and expects a result. For this, use the `startActivityForResult()` and `onActivityResult()` methods.

Transitioning between Activities with Intents

Android applications can have multiple entry points. A specific Activity can be designated as the main Activity to launch by default within the `AndroidManifest.xml` file; we talk more about this file in Chapter 5, “Defining the Manifest.”

Other activities might be designated to launch under specific circumstances. For example, a music application might designate a generic Activity to launch by default from the Application menu but also define specific, alternative entry-point activities for accessing specific music playlists by playlist ID or artists by name.

Launching a New Activity by Class Name

You can start activities in several ways. The simplest method is to use the application Context object to call the `startActivity()` method, which takes a single parameter, an Intent.

An Intent (`android.content.Intent`) is an asynchronous message mechanism used by the Android operating system to match task requests with the appropriate Activity or Service (launching it, if necessary) and to dispatch broadcast Intent events to the system at large.

For now, though, we focus on the Intent object and how it is used with activities. The following line of code calls the `startActivity()` method with an explicit Intent. This Intent requests the launch of the target Activity named `MyDrawActivity` by its class. This class is implemented elsewhere within the package.

```
startActivity(new Intent(getApplicationContext(),  
    MyDrawActivity.class));
```

This line of code might be sufficient for some applications, which simply transition from one Activity to the next. However, you can use the Intent mechanism in a much more robust manner. For example, you can use the Intent structure to pass data between activities.

Creating Intents with Action and Data

You've seen the simplest case of using an Intent to launch a class by name. Intents need not specify the component or class they want to launch explicitly. Instead, you can create an intent filter and register it within the Android manifest file. An intent filter is used by activities, services, and broadcast receivers to specify which intents each is interested in receiving (and filter out the rest). The Android operating system attempts to resolve the Intent requirements and launch the appropriate Activity based on the filter criteria.

The guts of the Intent object are composed of two main parts: the action to be performed and, optionally, the data to be acted upon. You can also specify action/data pairs using Intent action types and Uri objects. As you saw in Chapter 3, "Creating Your First Application," a Uri object represents a string that gives the location and name of an object. Therefore, an Intent is basically saying "do this" (the action) to "that" (the URI describing to what resource the action will be performed).

The most common action types are defined in the Intent class, including `ACTION_MAIN` (describes the main entry point of an Activity) and `ACTION_EDIT` (used in conjunction with a URI to the data edited). You will also find action types that generate integration points with activities in other applications, such as the browser or Phone Dialer.

Launching an Activity Belonging to Another Application

Initially, your application might launch only those activities defined within its own package. However, with the appropriate permissions, applications might also launch external activities within other applications. For example, a customer relationship management (CRM) application might launch the Contacts application to browse the Contacts database, choose a specific contact, and return that contact's unique identifier to the CRM application for use.

Here is an example of how to create a simple Intent with a predefined action (`ACTION_DIAL`) to launch the Phone Dialer with a specific phone number to dial in the form of a simple Uri object:


```
Uri number = Uri.parse("tel:5555551212");  
  
Intent dial = new Intent(Intent.ACTION_DIAL, number);  
  
startActivity(dial);
```

You can find a list of commonly used Google application intents at <http://d.android.com/guide/components/intents-common.html>. Also available is the developer-managed Registry of Intents protocols at OpenIntents, found at <http://www.openintents.org/>. A growing list of intents is available from third-party applications and those within the Android SDK.

Passing Additional Information Using Intents

You can also include additional data in an Intent. The extras property of an Intent is stored in a Bundle object. The Intent class also has a number of helper methods for getting and setting name/value pairs for many common data types.

For example, the following Intent includes two extra pieces of information—a String value and a Boolean:

```
Intent intent = new Intent(this, MyActivity.class);  
  
intent.putExtra("SomeStringData", "Foo");  
  
intent.putExtra("SomeBooleanData", false);  
  
startActivity(intent);
```

Then, in the onCreate() method of the MyActivity class, you can retrieve the extra data sent as follows:

Working with Services

Trying to wrap your head around activities and intents when you start with Android development can be daunting. We have tried to distill everything you need to know to start writing Android applications with multiple Activity or Fragment classes, but we'd be remiss if we didn't mention that there's a lot more here, much of which is discussed throughout this book using practical examples. However, we need to give you a "heads up" about some of these topics now because we begin to touch on them in the next chapter when we cover configuring the Android manifest file for your application.

One application component we have briefly discussed is the service. An Android Service (android.app.Service) can be thought of as a developer-created component that has no user interface of its own. An Android Service can be one of two things, or both. It can be used to perform lengthy operations that may go beyond the scope of a single Activity. Additionally, a Service can be the server of a client/server for providing functionality through remote invocation via interprocess communication (IPC). Although often used to control long-running server operations, the processing could be whatever the developer wants. Any Service classes exposed by an Android application must be registered in the Android manifest file.

You can use services for different purposes. Generally, you use a Service when no input is required from the user. Here are some circumstances in which you might want to implement or use an Android Service:

A weather, email, or social network app might implement a service to routinely check for updates on the network. (Note: There are other implementations for polling, but this is a common use of services.)

A game might create a service to download and process the content for the next level before the user actually needs it.

A photo or media app that keeps its data in sync online might implement a service to package and upload new content in the background when the device is idle.

A video-editing app might offload heavy processing to a queue on its service in order to avoid affecting overall system performance for nonessential tasks.

A news application might implement a service to “preload” content by downloading news stories before the user launches the application, to improve performance and responsiveness.

A good rule of thumb is that if the task requires the use of a worker thread, if it might affect application responsiveness and performance, and it is not time sensitive to the application, consider implementing a Service to handle the task outside the main application and any individual activity lifecycles.

TIP

In addition to deferring certain tasks with services, Android 5.0 Lollipop introduced the new JobScheduler API which allows you to schedule a Service to execute when certain conditions are met.

Receiving and Broadcasting Intents

Intents serve yet another purpose. You can broadcast an Intent (via a call to the `sendBroadcast()` method of the Context class) to the Android system at large, allowing any interested application (called a BroadcastReceiver) to receive that broadcast and act upon it. Your application might send off as well as listen for Intent broadcasts. Broadcasts are generally used to inform the system that something interesting has happened. For example, a commonly listened-for broadcast Intent is `ACTION_BATTERY_LOW`, which is broadcast as a warning when the battery is low. If your application has a battery-hogging service of some kind, or might lose data in the event of an abrupt shutdown, it might want to listen for this kind of broadcast and act accordingly. There are also broadcast events for other interesting system events, such as SD-card state changes, applications being installed or removed, and the wallpaper being changed.

Your application can also share information using this same broadcast mechanism. For example, an email application might broadcast an Intent whenever a new email arrives so that other applications (such as spam filters or antivirus apps) that might be interested in this type of event can react to it.

Quiz Questions

1. Which class does the Activity class extend?
2. What is the method for retrieving the application Context discussed in this chapter?

3. What is the method for retrieving application resources discussed in this chapter?
4. What is the method for accessing application preferences discussed in this chapter?
5. What is the method for retrieving application assets discussed in this chapter?
6. What is another name for the Activity stack?
7. What is the method for saving the Activity state discussed in this chapter?
8. What is the method for broadcasting an Intent discussed in this chapter?

Exercises

1. For each of the Activity callback methods, describe the method's overall purpose during the Activity lifecycle.
2. Using the online documentation, determine the method names responsible for the lifecycle of a Fragment.
3. Create a list of ten Activity action intents using the online documentation, where an Activity is the target component of the Intent action.
4. Using the online documentation, determine the method names responsible for the lifecycle of a Service.

Defining Your Application Using the Android Manifest File

Android projects use a special configuration file called the Android manifest file to determine application settings, such as the application name and version, as well as what permissions the application requires to run and what application components it is composed of. In this chapter, you explore the Android manifest file in detail and learn how applications use it to define and describe application behavior.

Configuring Android Applications Using the Android Manifest File

The Android application manifest file is a specially constructed XML file that must accompany each Android application. This file contains important information about the application's identity. Here, you define the application's name and version information as well as what application components the application relies on, what permissions the application requires to run, and other application configuration information.

The Android manifest file is named `AndroidManifest.xml`. The easiest way to find this file is by making the 1: Project tab the Active Tool Window of Android Studio and then selecting the Android tab from the drop down list (see Figure 5.1 left), so that the app folder is shown as the root for navigating your project structure. You should see a manifests folder inside the app folder that includes the `AndroidManifest.xml` file (see Figure 5.1, right).

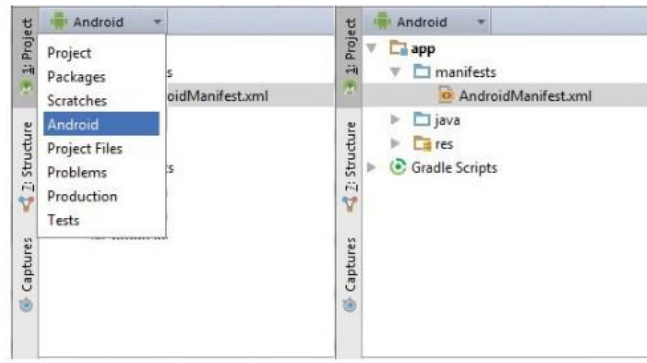


Figure 5.1 Selecting the Android tab from the Active Tool Window drop-down list (left) and showing the AndroidManifest.xml file inside the app/manifests directory (right).

The information in this file is used by the Android system to

- Install and upgrade the application package
- Display the application details to users, such as the application name, description, and icon
- Specify application system requirements, including which Android SDKs are supported, what device configurations are required (for example, D-pad navigation), and which platform features the application relies on (for example, multitouch capabilities)
- Specify what features are required by the application for app store-filtering purposes
- Register application activities and when they should be launched
- Manage application permissions
- Configure other advanced application component configuration details, including defining services, broadcast receivers, and content providers
- Specify intent filters for your activities, services, and broadcast receivers
- Enable application settings such as debugging and configuring instrumentation for application testing

TIP

When you create a project using Android Studio, the initial AndroidManifest.xml file is created for you. If you are using the android command-line tool, the Android manifest file is also created for you.

Editing the Android Manifest File

You can edit the Android manifest file, AndroidManifest.xml, by manually editing the XML. The Android manifest file is a specially constructed XML file.

Android manifest files include a single <manifest> tag with a single <application> tag. The following is a sample AndroidManifest.xml file for an application called Simple Hardware:

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
```

```
package="com.introtoandroid.simplehardware" >
<uses-permission android:name="android.permission.BATTERY_STATS" />
<uses-feature android:name="android.hardware.sensor.accelerometer" />
<uses-feature android:name="android.hardware.sensor.barometer" />
<uses-feature android:name="android.hardware.sensor.compass" />
<uses-feature android:name="android.hardware.sensor.gyroscope" />
<uses-feature android:name="android.hardware.sensor.light" />
<uses-feature android:name="android.hardware.sensor.proximity" />
<uses-feature android:name="android.hardware.sensor.stepcounter" />
<uses-feature android:name="android.hardware.sensor.stepdetector" />

<application
    android:allowBackup="true"
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name"
    android:theme="@style/AppTheme" >
    <activity
        android:name=".SimpleHardwareActivity"
        android:label="@string/app_name" >
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>
    <activity
        android:name=".SensorsActivity"
        android:label="@string/title_activity_sensors"
        android:parentActivityName=".SimpleHardwareActivity" >
        <meta-data
```

```

        android:name="android.support.PARENT_ACTIVITY"
        android:value="com.introtoandroid.simplehardware.
SimpleHardwareActivity" />
    </activity>
    <activity
        android:name=".BatteryActivity"
        android:label="@string/title_activity_battery"
        android:parentActivityName=".SimpleHardwareActivity" >
        <meta-data
            android:name="android.support.PARENT_ACTIVITY"
            android:value="com.introtoandroid.simplehardware.
SimpleHardwareActivity" />
    </activity>
</application>
</manifest>

```

NOTE

Within the previous manifest code listing, you may be wondering why there is a dot (.) in front of the <activity> tag name attributes. The dot is used as shorthand to specify that the SimpleHardware, SensorsActivity, and BatteryActivity classes belong to the package name specified in the manifest. We could have specified the entire package name path, but we have used the shorthand to save on typing the extra characters.

Here is a summary of what this file tells us about the SimpleHardware application:

- The application uses the package name com.introtoandroid.simplehardware.
- The application name and label are stored in the resource string called @string/app_name within the res/values/strings.xml resource file.
- The application theme is stored in the resource string called @style/AppTheme within the res/values/styles.xml/styles.xml files (there are two default files created for supporting devices' themes).
- The application supports backup and restore with allowBackup set to true.
- The application icon is the PNG file called ic_launcher.png stored within the res/mipmap/ic_launcher.png directory (there are actually multiple versions for different pixel densities).
- The application has four activities (MenuActivity, SimpleHardwareActivity, SensorsActivity, and BatteryActivity).

- SimpleHardwareActivity is the primary entry point for the application because it handles the action `android.intent.action.MAIN`. This Activity shows in the application launcher because its category is `android.intent.category.LAUNCHER`. Each `<activity>` tag also defines a name and a label attribute.
- SensorsActivity and BatteryActivity both have a `parentActivityName` of SimpleHardwareActivity, defined once with the `android:parentActivityName` attribute of the `<activity>` tag to support the Up navigation feature on API 17+, and once in the `<meta-data>` tag to support this same feature on API versions 4 to 16.
- The application requires the `BATTERY_STATS` permissions to run with the `<uses-permission>` tag.
- Finally, the application requests to use various different hardware sensors with the `<uses-feature>` tag.

TIP

When using the `<uses-feature>` tag, you can specify an optional attribute of `android:required` and set the value to true or false. This is used for configuring Google Play store filtering. If this attribute were set to true, Google Play would show your application listing only to users with devices that have that particular hardware or software feature, in this case, various sensors. To learn more about Google Play store filtering, visit <http://d.android.com/google/play/filters.html>.

Now let's talk about some of these important configurations in detail.

Managing Your Application's Identity

Your application's Android manifest file defines the application properties. The package name must be defined in the Android manifest file within the `<manifest>` tag using the `package` attribute:

```
<manifest
```

```
  xmlns:android="http://schemas.android.com/apk/res/android"
```

```
  package="com.introtoandroid.simplehardware">
```

Setting the Application Name and Icon

Overall application settings are configured with the `<application>` tag of the Android manifest file. Here, you set information such as the application icon (`android:icon`) and friendly name (`android:label`). These settings are attributes of the `<application>` tag.

For example, here we set the application icon to an image resource provided with the application package, and the application label to a string resource:

```
<application android:icon="@mipmap/ic_launcher"
```

```
  android:label="@string/app_name">
```

You can also set optional application settings as attributes in the <application> tag, such as the application description (android:description).

TIP

Android Marshmallow introduced an auto backup feature. To leverage this feature in your application, use the <application> attribute android:fullBackupContent and specify an XML file as the value that defines a data backup scheme your application should follow. This will help protect information your user cares about by backing up and restoring the information automatically, in case the user loses a device. To learn more about this feature and how to implement this functionality, see <http://d.android.com/preview/backup/index.html>.

Enforcing Application System Requirements

In addition to configuring your application's identity, the Android manifest file is used to specify any system requirements necessary for the application to run properly. For example, an augmented reality application might require that the device have GPS, a compass, and a camera.

These types of system requirements can be defined and enforced in the Android manifest file. When an application is installed on a device, the Android platform checks these requirements and will error out if necessary. Similarly, the Google Play store uses information in the Android manifest file to filter which applications to offer to which devices so that users install applications that should work on their devices.

Some of the application system requirements that developers can configure through the Android manifest file include:

- The Android platform features used by the application
- The Android hardware configurations required by the application
- The screen sizes and pixel densities supported by the application
- Any external libraries included in the application

Enforcing Application Platform Requirements

Android devices have different hardware and software configurations. Some devices have built-in hardware controls and others rely on the software only controls. Similarly, certain Android devices support the latest 3D graphics libraries and others provide little or no graphics support. The Android manifest file has several informational tags for flagging the system features and hardware configurations supported or required by an Android application.

Specifying Supported Input Methods

The <uses-configuration> tag can be used to specify which hardware and software input methods the application supports. There are different configuration attributes for five-way navigation: the hardware keyboard and keyboard types, navigation devices such as the directional pad, and touchscreen settings.

There is no “OR” support within a given attribute. If an application supports multiple input configurations, there must be multiple <uses-configuration> tags defined in your Android manifest file—one for each configuration supported.

For example, if your application requires a physical keyboard and touchscreen input using a finger or a stylus, you need to define two separate `<uses-configuration>` tags in your manifest file, as follows:

```
<uses-configuration android:reqHardKeyboard="true"
    android:reqTouchScreen="finger" />
<uses-configuration android:reqHardKeyboard="true"
    android:reqTouchScreen="stylus" />
```

For more information about the `<uses-configuration>` tag of the Android manifest file, see the Android SDK Reference at <http://d.android.com/guide/topics/manifest/uses-configuration-element.html>.

WARNING

Make sure to test your application with all of the available types of input methods, as not all devices support all types. For example, TVs do not have touchscreens, and if you design your application for touchscreen inputs, your app will not work properly on TV devices.

Specifying Required Device Features

Not all Android devices support every Android feature. Put another way, there are a number of APIs (and related hardware) that Android device manufacturers and carriers may optionally include. For example, not all Android devices have multitouch capability or a camera flash.

The `<uses-feature>` tag is used to specify which Android features your application uses to run properly. These settings are for informational purposes only—the Android operating system does not enforce these settings, but publication channels such as the Google Play store use this information to filter the applications available to a given user. Other applications might check this information as well.

If your application requires multiple features, you must create a `<uses-feature>` tag for each feature. For example, an application that requires both a light and a proximity sensor requires two tags:

```
<uses-feature android:name="android.hardware.sensor.light" />
<uses-feature android:name="android.hardware.sensor.proximity" />
```

One common reason to use the `<uses-feature>` tag is for specifying the OpenGL ES versions supported by your application. By default, all applications function with OpenGL ES 1.0 (which is a required feature of all Android devices). However, if your application requires features available only in later versions of OpenGL ES, such as 2.0 or 3.0, you must specify this feature in the Android manifest file. This is done using the `android:glEsVersion` attribute of the `<uses-feature>` tag. Specify the lowest version of OpenGL ES that the application requires. If the application works with 1.0, 2.0, and 3.0, specify the lowest version (so that the Google Play store allows more users to install your application).

For more information about the `<uses-feature>` tag of the Android manifest file, see the Android SDK Reference at <http://d.android.com/guide/topics/manifest/uses-feature-element.html>.

TIP

If a certain feature is not required for your application to function properly, rather than providing filters for the Google Play store to limit access to specific devices, you could check for certain device features at runtime and allow specific application functionality only if those particular features are present on the user's device. That way, you maximize the number of users who can install and use your application. To check for specific features at runtime, use the `hasSystemFeature()` method. For example, to see if the device your application is running on has touchscreen capabilities and returns a Boolean value:

```
getPackageManager().hasSystemFeature("android.hardware.touchscreen");
```

Specifying Supported Screen Sizes

Android devices come in many shapes and sizes. Screen sizes and pixel densities vary tremendously across the wide range of Android devices available on the market today. The `<supports-screens>` tag can be used to specify which Android screen types the application supports. The Android platform categorizes screen types in terms of sizes (small, normal, large, and xlarge) and pixel density (LDPI, MDPI, HDPI, XHDPI, XXHDPI, and XXXHDPI, representing low-, medium-, high-, extra-high-, extra-extra-high-, and extra-extra-extra-high-density displays). These characteristics effectively cover the variety of screen types available within the Android platform.

For example, if the application supports small screens and normal screens regardless of pixel density, the application's `<supports-screens>` tag is configured as follows:

```
<supports-screens android:resizable="false"
    android:smallScreens="true"
    android:normalScreens="true"
    android:largeScreens="false"
    android:xlargeScreens="false"
    android:compatibleWidthLimitDp="320"
    android:anyDensity="true"/>
```

For more information about the `<supports-screens>` tag of the Android manifest file, see the Android SDK Reference at <http://d.android.com/guide/topics/manifest/supports-screens-element.html> as well as the Android Developers Guide on screen support at http://d.android.com/guide/practices/screens_support.html#DensityConsiderations.

Other Application-Configuration Settings and Filters

You'll want to be aware of several other lesser-used manifest file settings because they are also used by the Google Play store for application filtering:

The `<supports-gl-texture>` tag is used to specify the GL texture compression format supported by the application. This tag is used by applications that use the graphics libraries and are intended to be compatible only with devices that support a specific compression format. For more information about this manifest file tag, see the Android SDK documentation at <http://d.android.com/guide/topics/manifest/supports-gl-texture-element.html>.

The `<compatible-screens>` tag is used solely by the Google Play store to restrict installation of your application to devices with specific screen sizes. This tag is not checked by the Android operating system, and usage is discouraged unless you absolutely need to restrict the installation of your application on certain devices. For more information about this manifest file tag, see the Android SDK documentation at <http://d.android.com/guide/topics/manifest/compatible-screens-element.html>.

Registering Activities in the Android Manifest

Each Activity within the application must be defined within the Android manifest file with an `<activity>` tag. For example, the following XML excerpt registers an Activity class called `SensorActivity`:

```
<activity android:name="SensorsActivity" />
```

This Activity must be defined as a class within the `com.introtoandroid.simplehardware` package—that is, the package specified in the `<manifest>` element of the Android manifest file. You can also enforce the scope of the Activity class by using the dot as a prefix in the Activity class name:

```
<activity android:name=".SensorsActivity" />
```

Or, you can specify the complete class name:

```
<activity android:name="com.introtoandroid.simplehardware.SensorsActivity" />
```

WARNING

You must define the `<activity>` tag for each Activity or it will not run as part of your application. It is quite common for developers to implement an Activity and then forget to do this. They then spend a lot of time troubleshooting why it isn't running properly, only to realize they forgot to register it in the Android manifest file.

Designating a Primary Entry-Point Activity for Your Application Using an Intent Filter

You designate an Activity class as the primary entry point by configuring an intent filter using the Android manifest tag `<intent-filter>` in the Android manifest file with the `MAIN` action type and the `LAUNCHER` category.

For example, the following XML configures an Activity called `SimpleHardwareActivity` as the primary launching point of the application:

```
<activity android:name=".SimpleHardwareActivity"
    android:label="@string/app_name">
    <intent-filter>
```

```
<action android:name="android.intent.action.MAIN" />

<category android:name="android.intent.category.LAUNCHER" />

</intent-filter>

</activity>
```

Configuring Other Intent Filters

The Android operating system uses intent filters to resolve implicit intents—that is, intents that do not specify a specific activity or other component type to launch. Intent filters can be applied to activities, services, and broadcast receivers. An intent filter declares that this application component is capable of handling or processing a specific type of intent when it matches the filter’s criteria.

Different applications have the same sorts of intent filters and are able to process the same sorts of requests. In fact, this is how the “share” features and the flexible application launch system of the Android operating system work. For example, you can have several different Web browsers installed on a device, all of which can handle “browse the Web” intents by setting up the appropriate filters.

Intent filters are defined using the `<intent-filter>` tag and must contain at least one `<action>` tag, but can also contain other information such as `<category>` and `<data>` blocks. Here, we have a sample intent filter block that might be found within an `<activity>` block:

```
<intent-filter>

  <action android:name="android.intent.action.VIEW" />

  <category android:name="android.intent.category.BROWSABLE" />

  <category android:name="android.intent.category.DEFAULT" />

  <data android:scheme="geoname"/>

</intent-filter>
```

This intent filter definition uses a predefined action called VIEW, the action for viewing particular content. It also handles Intent objects in the BROWSABLE or DEFAULT category, and uses a scheme of geoname so that when a URI starts with geoname://, the Activity with this intent filter can be launched to view the content.

TIP

You can define custom actions unique to your application. If you do so, be sure to document these actions if you want them to be used by third parties. You can document these however you want: your SDK documentation could be provided on your website, or confidential documents could be given directly to a client.

Registering Other Application Components

All application components must be defined within the Android manifest file. In addition to activities, all services and broadcast receivers must be registered within the Android manifest file.

Services are registered using the `<service>` tag.

Broadcast receivers are registered using the `<receiver>` tag.

Content providers are registered using the `<provider>` tag.

Both services and broadcast receivers use intent filters. If your application acts as a content provider, effectively exposing a shared data service for use by other applications, it must declare this capability within the Android manifest file using the `<provider>` tag. Configuring a content provider involves determining what subsets of data are shared and what permissions are required to access them, if any. We begin our discussion of content providers in Chapter 17, “Leveraging Content Providers.”

TIP

Many of the code examples provided in this chapter are taken from the SimplePermissions application. The source code for this application is provided for download on the book’s website (<http://introductiontoandroid.blogspot.com>).

Working with Permissions

The Android operating system has been locked down so that applications have limited capability to adversely affect operations outside their process space. Instead, Android applications run within the bubble of their own virtual machine, with their own Linux user account (and related permissions).

Registering Permissions Your Application Requires

Android applications have no permissions by default. Instead, any permissions for shared resources or privileged access—whether it’s shared data, such as the Contacts database, or access to underlying hardware, such as the built-in camera—must be explicitly registered within the Android manifest file. For devices running versions of Android prior to Marshmallow 6.0 API Level 23, these permissions are granted when the application is installed. For devices running Android Marshmallow 6.0 API Level 23 and newer, permissions with a level of `PROTECTION_NORMAL`, and some with `PROTECTION_SIGNATURE`, are granted at installation, while those with a `PROTECTION_DANGEROUS` must be requested and verified at runtime.

The following XML excerpt for inclusion in the Android manifest file defines a permission using the `<uses-permission>` tag to read from and write to the contact database, both having a permission level of `PROTECTION_DANGEROUS`:

```
<uses-permission android:name="android.permission.READ_CONTACTS" />
```

```
<uses-permission android:name="android.permission.WRITE_CONTACTS" />
```

A complete list of the permissions can be found in the `android.Manifest.permission` class. Your application manifest should include only the permissions required to run.

TIP

You might find that, in certain cases, permissions are not enforced (you can operate without the permission) by one device or another. In these cases, it is prudent to request the permission anyway for two reasons. First, the user is informed that the application is performing those sensitive actions, and second, that permission could be enforced in a later device update. Also be aware that in early SDK versions, not all permissions were necessarily enforced at the platform level.

WARNING

If the application description or type of application you are providing does not clearly justify the permissions requested, you may get low ratings simply for requesting unnecessary permissions. We see many applications asking for permissions they do not need or have no reason to request. Many people who realize this will not follow through and install the application. Privacy is a big concern for many users, so be sure to respect it.

Requesting Permissions at Runtime

Android Marshmallow introduced a new permissions model allowing users to install your application and to accept your applications permissions once they interact with the features that require them. This new permissions model is important because it reduces the amount of friction permissions may have caused in the past, such as users abandoning the installation of your application because they are not comfortable accepting a particular permission.

With the Android Marshmallow permissions model, a user does not need to grant permissions prior to installing, allowing him or her to install and start interacting with your application. Once the user comes across a feature that requires a particular permission, he or she is presented with a dialog requesting that the permission be granted. If the permission is granted, the system notifies the application and permission is then granted to the user. If the permission is not granted, the user will not be able to access that particular functional area of your application that requires the denied permission.

Permissions are declared as normal with the `<uses-permission>` tag in the Android manifest file. Your applications code is where you check to see if the permission has been granted. This permissions model allows users to revoke access to particular permissions in the application settings, without having to uninstall your application. Even if a user grants a particular permission, he or she is able to revoke permissions at any time, so your application must always check that a particular permission is granted, and if not, make a request for the permission.

To request permissions at runtime, you must first add the following dependencies to your build.gradle app module file (we talk more about Gradle and dependencies in Appendix E, “Quick-Start: Gradle Build System”):

```
compile 'com.android.support:support-v4:23.0.0'
```

```
compile 'com.android.support:appcompat-v7:23.0.0'
```

Then your Activity must extend from AppCompatActivity and implement the ActivityCompat.OnRequestPermissionsResultCallback interface as shown here:

```
public class PermissionsActivity extends AppCompatActivity  
    implements ActivityCompat.OnRequestPermissionsResultCallback {  
    // Activity code here  
}
```

Then you must check if the permission has been granted yet, and if not, request the permission. This is performed as shown below:

```
if (ActivityCompat.checkSelfPermission(this, Manifest.permission.READ_CONTACTS)  
    != PackageManager.PERMISSION_GRANTED  
  
    || ActivityCompat.checkSelfPermission(this, Manifest.permission.WRITE_  
CONTACTS)  
    != PackageManager.PERMISSION_GRANTED) {  
    Log.i(DEBUG_TAG, "Contact permissions not granted. Requesting permissions.");  
    ActivityCompat.requestPermissions(GridListMenuActivity.this, {  
        Manifest.permission.READ_CONTACTS,  
        Manifest.permission.WRITE_CONTACTS}, 0);  
} else {  
    Log.i(DEBUG_TAG,  
        "Contact permissions granted. Displaying contacts.");  
    // Do work here  
}
```

This if statement checks if the READ_CONTACTS and WRITE_CONTACTS permissions have been granted, and if not, calls the requestPermissions() method of ActivityCompat. You then must implement the OnRequestPermissionsResultCallback by overriding the onRequestPermissionsResult() method in your Activity as shown below:

```
@Override  
public void onRequestPermissionsResult(int requestCode,  
    @NonNull String[] permissions, @NonNull int[] grantResults) {  
    if (requestCode == REQUEST_CONTACTS) {
```

```

    Log.d(DEBUG_TAG, "Received response for contact permissions request.");
    // All Contact permissions must be checked
    if (verifyPermissions(grantResults)) {
        // All required permissions granted, proceed as usual
        Log.d(DEBUG_TAG, "Contacts permissions were granted.");
        Toast.makeText(this, "Contacts Permission Granted",
            Toast.LENGTH_SHORT).show();
    } else {
        Log.d(DEBUG_TAG, "Contacts permissions were denied.");
        Toast.makeText(this, "Contacts Permission Denied",
            Toast.LENGTH_SHORT).show();
    }
} else {
    super.onRequestPermissionsResult(requestCode, permissions, grantResults);
}
}

```

This method handles the result input by the user, and if the user accepted, the permission is now available to your application; if the user denied the request, the permission will not be available. Whatever features you develop that require those denied permissions will not be made available until the user chooses to grant the appropriate permission.

NOTE

Android Marshmallow introduced a permission for fingerprint authentication. To add this permission to your applications manifest, use the `android.permission.USE_FINGERPRINT` value. This will allow your application to support fingerprint authentication on devices that have the appropriate hardware support.

Registering Permissions Your Application Enforces

Applications can also define and enforce their own permissions via the `<permission>` tag to be used by other applications. Permissions must be described and then applied to specific application components, such as activities, using the `android:permission` attribute.

TIP

Use Java-style scoping for unique naming of application permissions (for example, `com.introtoandroid.media.ViewMatureMaterial`).

Permissions can be enforced at several points:

- When starting an Activity or Service
- When accessing data provided by a content provider
- At the method call level
- When sending or receiving broadcasts by an Intent

Permissions can have three primary protection levels: normal, dangerous, and signature. The normal protection level is a good default for fine-grained permission enforcement within the application. The dangerous protection level is used for higher-risk activities, which might adversely affect the device. Finally, the signature protection level permits any application signed with the same certificate to use that component for controlled application interoperability. You will learn more about application signing in Chapter 22, “Distributing Your Applications.”

Permissions can be broken down into categories, called permission groups, which describe or warn why specific activities require permission. For example, permissions might be applied for activities that expose sensitive user data such as location and personal information (`android.permission-group.LOCATION` and `android.permission-group.PERSONAL_INFO`), access underlying hardware (`android.permission-group.HARDWARE_CONTROLS`), or perform operations that might incur fees to the user (`android.permission-group.COST_MONEY`). A complete list of permission groups is available within the `Manifest.permission_group` class.

For more information about applications and how they can enforce their own permissions, check out the `<permission>` manifest tag SDK documentation at <http://d.android.com/guide/topics/manifest/permission-element.html>.

Exploring Other Manifest File Settings

We have now covered the basics of the Android manifest file, but many other settings are configurable within the Android manifest file using different tag blocks, not to mention attributes within each tag that we have already discussed.

Some other features you can configure within the Android manifest file include

- Setting application-wide themes within the `<application>` tag attributes
- Configuring unit-testing features using the `<instrumentation>` tag
- Aliasing activities using the `<activity-alias>` tag
- Creating broadcast receivers using the `<receiver>` tag
- Creating content providers using the `<provider>` tag, along with managing content-provider permissions using the `<grant-uri-permission>` and `<path-permission>` tags
- Including other data within your activity, service, or receiver-component registrations with the `<meta-data>` tag

For more detailed descriptions of each tag and attribute available in the Android SDK (and there are many), please review the Android SDK Reference on the Android manifest file at <http://d.android.com/guide/topics/manifest/manifest-intro.html>.

Summary

Each Android application has a specially formatted XML configuration file called `AndroidManifest.xml`. This file describes the application's identity in great detail. Some information that you must define within the Android manifest file includes the application's package and name information, what application components it contains, which device configurations it requires, and what permissions it needs to run. The Android manifest file is used by the Android operating system to install, upgrade, and run the application package. Some details of the Android manifest file are also used by third parties, including the Google Play publication channel. In addition, the new Android Marshmallow permission model allows users to grant access to permissions at runtime and revoke access to particular permissions.

Quiz Questions

1. What is the manifest XML tag for specifying the input methods your application supports?
2. What is the manifest XML tag for specifying the required device features for your application?
3. What is the manifest XML tag for specifying screen sizes supported by your application?
4. What is the manifest XML tag for registering permissions that your application enforces?
5. What is the manifest XML `<uses-permission>` attribute for supporting fingerprint authentication?

Exercises

1. Define a fictitious `<application>` manifest XML tag; include the `icon`, `label`, `allowBackup`, `enabled`, and `testOnly` attributes; and then include values for each.
2. Using the Android documentation, list all the potential string values available for defining the `reqNavigation` attribute of the `<uses-configuration>` tag.
3. Using the Android documentation, create a list of five hardware features of the `name` attribute of the `<uses-feature>` tag.
4. Using the Android documentation, name all of the possible attributes and their value types of the `<supports-screens>` manifest XML tag.
5. Using the Android documentation, create a list of ten different values that could be used for defining the `name` attribute of the `<uses-permission>` manifest XML tag.

Managing Application Resources

The well-written application accesses its resources programmatically instead of the developer hard-coding them into the source code. This is done for a variety of reasons. Storing application resources in a single place is a more organized approach to development and makes the code more readable and

maintainable. Externalizing resources such as strings makes it easier to localize applications for different languages and geographic regions. Finally, different resources may be necessary for different devices.

In this chapter, you learn how Android applications store and access important resources such as strings, graphics, and other data. You also learn how to organize Android resources within the project files for localization and different device configurations.

What Are Resources?

All Android applications are composed of two things: functionality (code instructions) and data (resources). The functionality is the code that determines how your application behaves. This includes any algorithms that make the application run. Resources include text strings, styles and themes, dimensions, images and icons, audio files, videos, and other data used by the application.

TIP

Many of the code examples provided in this chapter are taken from the SimpleResourceView, ResourceRoundup, and ParisView applications. The source code for these applications is provided for download on the book’s website (<http://introductiontoandroid.blogspot.com>).

Storing Application Resources

Android resource files are stored separately from the .java class files in the Android project. Most common resource types are stored in XML format. You can also store raw data files and graphics as resources. Resources are organized in a strict directory hierarchy. All resources must be stored under the res project directory in specially named subdirectories whose names must be lowercase.

Different resource types are stored in different directories. The resource subdirectories generated when you create an Android project are shown in Table 6.1.

Resource Subdirectory	Purpose
res/drawable/	Graphics resources
res/layout/	User interface resources
res/menu/	Menu resources for showing options or actions in activities
res/mipmap/	App launcher icon resources
res/values/	Simple data such as strings, styles and themes, and dimensions

Table 6.1 Default Android Resource Directories

Table 6.1 Default Android Resource Directories

Each resource type corresponds to a specific resource subdirectory name. For example, all graphics are stored under the `res/drawable` directory structure. Resources can be further organized in a variety of ways using even more specially named directory qualifiers. For example, a `res/drawable-hdpi` directory would be used to store graphics for high-density screens; a `res/drawable-ldpi` directory is used to store graphics for low-density screens; a `res/drawable-mdpi` directory stores graphics for medium-density screens; a `res/drawable-xhdpi` directory stores graphics for extra-high-density screens; and a `res/drawable-xxhdpi` directory stores graphics for extra-extra-high-density screens. If you had a graphics resource that was shared by all screens, you would simply store that resource in the `res/drawable` directory. We talk more about resource directory qualifiers later in this chapter.

If you use the Android Studio, you will find that adding resources to your project is simple. Android Studio automatically detects new resources when you add them to the appropriate project resource subdirectory under `res`. These resources are compiled, resulting in the generation of the `R.java` source file, which enables you to access your resources programmatically.

Storing Primitive Resource Types

Simple resource value types, such as strings, colors, dimensions, and other primitives, are stored under the `res/values` project directory in XML files. Each resource file under the `res/values` directory should begin with the following XML header:

```
<?xml version="1.0" encoding="utf-8"?>
```

Next comes the root node `<resources>` followed by the specific resource element types such as `<string>` or `<color>`. Each resource is defined using a different element name. Primitive resource types simply have a unique name and a value, like this color resource:

```
<color name="myFavoriteShadeOfRed">#800000</color>
```

TIP

Although the XML filenames are arbitrary, the best practice is to store your resources in separate files to reflect their types, such as `strings.xml`, `colors.xml`, and so on. However, there is nothing stopping developers from creating multiple resource files for a given type, such as two separate XML files called `bright_colors.xml` and `muted_colors.xml`, if they so choose. You will learn later in Chapter 13, “Designing Compatible Applications,” about alternative resources that also influence how the files may be named and subdivided.

Storing Graphics and Files

In addition to simple resource types stored in the `res/values` directory, you can also store numerous other types of resources, such as graphics, arbitrary XML files, and raw files. These types of resources are not stored in the `res/values` directory but instead are stored in specially named directories according to their type. For example, graphics are stored as files in the `res/drawable` directory structure. XML files can be stored in the `res/xml` directory, and raw files can be stored in the `res/raw` directory.

Make sure you name resource files appropriately because the resource name for graphics and files is derived from the filename of the specific resource. For example, a file called `flag.png` in the `res/drawable` directory is given the name `R.drawable.flag`.

Storing Other Resource Types

All other resource types—whether they are tween animation sequences, color state lists, or menus—are stored in special XML formats in various directories, as shown in Table 6.2. Again, each resource must be uniquely named.

Understanding How Resources Are Resolved

The Android platform has a very robust mechanism for loading the appropriate resources at runtime. You can organize Android project resources based on more than a dozen different criteria. It can be useful to think of the resources stored in the directory hierarchy discussed in this chapter as the application's default resources. Under certain conditions, you can also supply special versions of your resources to load instead of the defaults. These specialized resources are called alternative resources.

Two common reasons that developers use alternative resources are for internationalization and localization purposes, and to design an application that runs smoothly on different device screens and orientations. We focus on default resources in this chapter and discuss alternative resources in Chapter 13, "Designing Compatible Applications."

Default and alternative resources are best illustrated by example. Let's presume that we have a simple application with its requisite string, graphics, and layout resources. In this application, the resources are stored in the top-level resource directories (for example, `res/values/strings.xml`, `res/drawable/mylogo.png`, and `res/layout/main.xml`). No matter what Android device (huge hi-def screen, postage-stamp-size screen, portrait or landscape orientation, and so on) you run this application on, the same resource data is loaded and used. This application uses only default resources.

But what if we want our application to use different graphics sizes based on the screen density? We could use alternative graphics resources to do this. For example, we could provide different logos for different device screen densities by providing many versions of `mylogo.png`:

- `res/drawable-ldpi/mylogo.png` (low-density screens)
- `res/drawable-mdpi/mylogo.png` (medium-density screens)
- `res/drawable-hdpi/mylogo.png` (high-density screens)
- `res/drawable-xhdpi/mylogo.png` (extra-high-density screens)
- `res/drawable-xxhdpi/mylogo.png` (extra-extra-high-density screens)
- `res/drawable-xxxhdpi/mylogo.png` (extra-extra-extra-high-density screens)
- `res/drawable-nodpi/mylogo.png` (do not scale for any screens)
- `res/drawable-tvdpi/mylogo.png` (between medium-and high-density screens)

Let's look at another example. Let's say we find that the application would look much better if the layout were fully customized for portrait versus landscape orientation. We could change the layout, moving controls around, in order to achieve a more pleasant user experience and provide two layouts:

- `res/layout-port/main.xml` (layout loaded in portrait mode)
- `res/layout-land/main.xml` (layout loaded in landscape mode)

We are introducing the concept of alternative resources now because they are hard to avoid completely, but we will work primarily with default resources for most of this book, simply in order to focus on specific programming tasks without the clutter that results from trying to customize an application to run beautifully on every device configuration one might use.

Accessing Resources Programmatically

Developers access specific application resources using the `R.java` class file and its subclasses, which are automatically generated when you add resources to your project (if you use Android Studio). You can refer to any resource identifier in your project by its name (which is why it must be unique). For example, a `String` named `strHello` defined within the resource file called `res/values/strings.xml` is accessed in the code as follows:

```
R.string.strHello
```

This variable is not the actual data associated with the `String` named `strHello`. Instead, you use this resource identifier to retrieve the resource of that type (which happens to be `String`) from the project resources associated with the application.

First, you retrieve the `Resources` instance for your application `Context` (`android.content.Context`), which is, in this case, this because the `Activity` class extends `Context`. Then you use the `Resources` instance to get the appropriate kind of resource you want. You find that the `Resources` class (`android.content.res.Resources`) has helper methods for handling every kind of resource.

For example, a simple way to retrieve the `String` text is to call the `getString()` method of the `Resources` class, like this:

```
String myString = getResources().getString(R.string.strHello);
```

Before we go any further, we find it can be helpful to dig in and create some resources, so let's create a simple example.

Adding Simple Resource Values in Android Studio

To illustrate how to add resources in Android Studio, let's look at an example. Create a new Android project and navigate to the `res/values/strings.xml` file in the Android Studio and double-click the file to edit it. Alternatively, you can use the Android project included with the book called `ResourceRoundup` to follow along. Your `strings.xml` resource file opens in the editor and should look something like Figure 6.1, but with fewer strings.

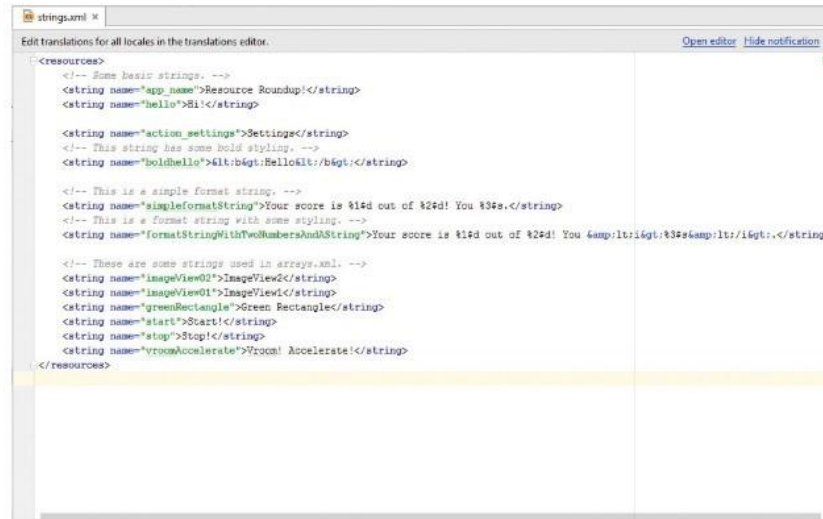


Figure 6.1 A sample string resource file in the Android Studio editor (XML).

Now add some resources to the XML. Specifically, create the following resources:

- A color resource named `prettyTextColor` with a value of `#ff0000`
- A dimension resource named `textPointSize` with a value of `14pt`
- A drawable resource named `redDrawable` with a value of `#F00`

Now you have several resources of various types in your strings.xml resource file, which now should look something like this:

```
<?xml version="1.0" encoding="utf-8"?>
```

<resources>

```
<string name="app_name">ResourceRoundup</string>
```

<string

```
name="hello">Hello World, ResourceRoundupActivity</string>
```

```
<color name="prettyTextColor">#ff0000</color>
```

```
<dimen name="textPointSize">14pt</dimen>
```

```
<drawable name="redDrawable">#F00</drawable>
```

</resources>

Save the `strings.xml` resource file. Android Studio automatically generates the `R.java` file in your project, with the appropriate resource IDs, which enables you to access your resources programmatically after they are compiled into the project. You may navigate to your `R.java` file by switching the Active Tool Window of the Project tab of Android Studio to the Project Files selection. Then expand the `app` folder so that the build folder is visible. Then expand further into `r/debug`, and finally expand into `com/introtoandroid/samples/resourceroundup`, and you should see the `R.java` file listing (see Figure 6.2).

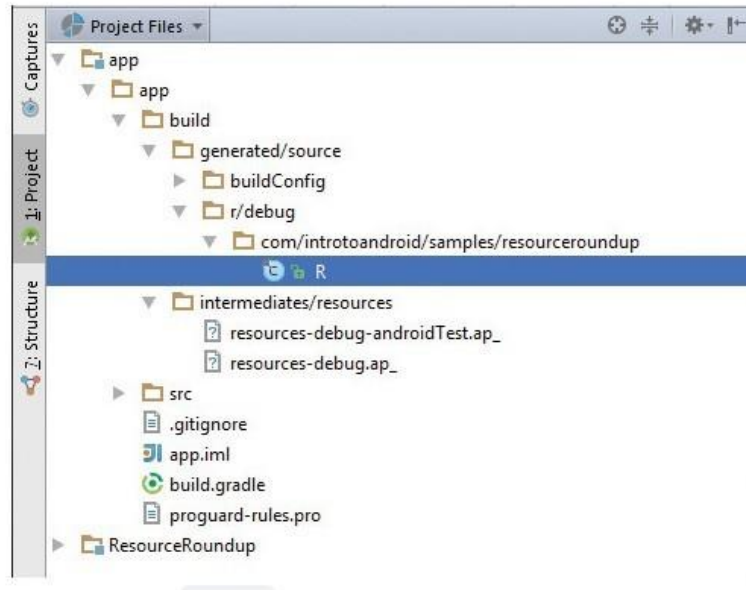


Figure 6.2 Showing the R.java file within Android Studio.

Double-click to open the R.java file, which should look something like this:

```
package com.introtoandroid.resourceroundup;

public final class R {

    public static final class attr {

    }

    public static final class color {

        public static final int prettyTextColor=0x7f050000;

    }

    public static final class dimen {

        public static final int textPointSize=0x7f060000;

    }

    public static final class drawable {

        public static final int icon=0x7f020000;

        public static final int redDrawable=0x7f020001;

    }

    public static final class layout {

        public static final int main=0x7f030000;
```



```

    }

    public static final class string {

        public static final int app_name=0x7f040000;

        public static final int hello=0x7f040001;

    }

}

```

Now you are free to use these resources in your code. If you navigate to your ResourceRoundupActivity.java source file, you can add some lines to retrieve your resources and work with them, like this:

```

String myString = getResources().getString(R.string.hello);

int myColor =

    ContextCompat.getColor(context, R.color.prettyTextColor);

float myDimen =

    getResources().getDimension(R.dimen.textPointSize);

ColorDrawable myDraw = (ColorDrawable) ContextCompat.

    getDrawable(R.drawable.redDrawable);

```

If we go back to the strings.xml file, we can add a string array to our resource listing by adding the following XML element:

```

<?xml version="1.0" encoding="utf-8"?>

<resources>

    <string name="app_name">Use Some Resources</string>

    <string

        name="hello">Hello World, UseSomeResources</string>

    <color name="prettyTextColor">#ff0000</color>

    <dimen name="textPointSize">14pt</dimen>

    <drawable name="redDrawable">#F00</drawable>

    <string-array name="flavors">

        <item>Vanilla</item>

        <item>Chocolate</item>

```

```

        <item>Strawberry</item>

    </string-array>

</resources>

```

Save the strings.xml file and now the string array named flavors is available in your source file R.java, so you can use it programmatically in ResourceRoundupActivity.java, like this:

```

String[] aFlavors =

    getResources().getStringArray(R.array.flavors);

```

You now have a general idea of how to add simple resources using the Android Studio editor, but there are quite a few different types of data available to add as resources. It is a common practice to store different types of resources in different files. For example, you might store the strings in res/values/strings.xml, but store the prettyTextColor color resource in res/values/colors.xml and the textPointSize dimension resource in res/values/dimens.xml. Reorganizing where you keep your resources in the resource directory hierarchy does not change the names of the resources or the code used earlier to access the resources programmatically.

Now let's take a closer look at how to add some of the most common types of resources to your Android applications.

Working with Different Types of Resources

In this section, we look at the specific types of resources available for Android applications, how they are defined in the project files, and how you can access this resource data programmatically. For each type of resource, you learn what types of values can be stored and in what format.

Working with String Resources

String resources are among the simplest resource types available to the developer. String resources might be used to show text labels on form views and for help text. The application name is also stored as a string resource, by default.

String resources are defined in XML under the res/values project directory and compiled into the application package at build time. All strings with apostrophes or single straight quotes need to be escaped or wrapped in double straight quotes. Some examples of well-formatted string values are shown in Table 6.3.

String Resource Value	Displays As
Hello, World	Hello, World
"User's Full Name:"	User's Full Name:
User\'s Full Name:	User's Full Name:
She said, \"Hi.\"	She said, "Hi."
She\'s busy but she did say, \"Hi.\"	She's busy but she did say, "Hi."

Table 6.3 String Resource Formatting Examples

You can edit the XML by double-clicking the file and opening it in the Android Studio editor. After you save the file, the resource identifiers are automatically added to your R.java class file.

String values are appropriately tagged with the `<string>` tag and represent a name/value pair. The name attribute is how you refer to the specific string programmatically, so name these resources wisely.

Here is an example of the string resource file `res/values/strings.xml`:

```
<?xml version="1.0" encoding="utf-8"?>

<resources>

    <string name="app_name">Resource Viewer</string>

    <string name="test_string">Testing 1,2,3</string>

    <string name="test_string2">Testing 4,5,6</string>

</resources>
```

Bold, Italic, and Underlined Strings

You can also add three HTML-style attributes to string resources: bold, italic, and underlining. You specify the styling using the `<b>`, `<i>`, and `<u>` tags, respectively; for example:

```
<string
    name="txt"><b>Bold</b>,<i>Italic</i>,<u>Line</u></string>
```

Using String Resources as Format Strings

You can create format strings, but you need to escape all bold, italic, and underlining tags if you do so. For example, this text shows a score and the “win” or “lose” string:

```
<string
    name="winLose">Score: %1$d of %2$d! You %3$s.</string>
```

If you want to include bold, italic, or underlining in this format string, you need to escape the format tags. For example, if you want to italicize the “win” or “lose” string at the end, your resource would look like this:

```
<string name="winLoseStyled">
    Score: %1$d of %2$d! You &lt;i&gt;%3$s&lt;/i&gt;.</string>
```

NOTE

Those of you who are familiar with XML will recognize this as standard XML escaping. Indeed, that’s all it is. After the standard set of XML escape characters is parsed, the string is then interpreted with the formatting tags. As with any XML document, you’d also need to escape single quotes (`'` is `'`), double quotes (`"` is `"`), and ampersands (`&` is `&`).

Using String Resources Programmatically

As shown earlier in this chapter, accessing string resources in code is straightforward. There are two primary ways in which you can access a string resource.

The following code accesses your application's string resource named `hello`, returning only the `String`. All HTML-style attributes (bold, italic, and underlining) are stripped from the `String`.

```
String myStrHello =  
    getResources().getString(R.string.hello);
```

You can also access the `String` and preserve the formatting by using this alternative method:

```
CharSequence myBoldStr =  
    getResources().getText(R.string.boldhello);
```

To load a format `String`, you need to make sure any format variables are properly escaped. One way you can do this is by using the `htmlEncode()` method of the `TextUtils` (`android.text.TextUtils`) class:

```
String mySimpleWinString;  
mySimpleWinString =  
    getResources().getString(R.string.winLose);  
String escapedWin = TextUtils.htmlEncode(mySimpleWinString);  
String resultText = String.format(mySimpleWinString, 5, 5, escapedWin);
```

The resulting text in the `resultText` variable is

Score: 5 of 5! You Won.

Now, if you have styling in this format `String` like the preceding string resource `winLoseStyled`, you need to take a few more steps to handle the escaped italic tags. For this, you might want to use the `fromHtml()` method of the `Html` class (`android.text.Html`), as shown here:

```
String myStyledWinString;  
myStyledWinString =  
    getResources().getString(R.string.winLoseStyled);  
String escapedWin = TextUtils.htmlEncode(myStyledWinString);  
String resultText =  
    String.format(myStyledWinString, 5, 5, escapedWin);  
CharSequence styledResults = Html.fromHtml(resultText);
```

The resulting text in the `styledResults` variable is

Score: 5 of 5! You *<i>Won</i>*.

This variable, `styledResults`, can then be used in user interface controls such as `TextView` objects, where styled text is displayed correctly.

Working with Quantity Strings

A special resource type called `<plurals>` can be used to define strings that are useful for changing a word's grammatical quantity form. Here is an example string resource file with the resource path of `res/values/strings.xml` that defines two different quantity forms of a particular animal name that changes based on the context of the quantity:

```
<resources>

    <plurals name="quantityOfGeese">

        <item quantity="one">You caught a goose!</item>

        <item quantity="other">You caught %d geese!</item>

    </plurals>

</resources>
```

The singular form for this particular animal is `goose`, and the plural form is `geese`. The `%d` value is used so we can display the exact quantity of geese to the user. To work with pluralized resources in your code, the method `getQuantityString()` can be used to retrieve a plural string resource as shown here:

```
int quantity = getQuantityOfGeese();

Resources plurals = getResources();

String geeseFound = plurals.getQuantityString(

    R.plurals.quantityOfGeese, quantity, quantity);
```

`getQuantityString()` takes three variables. The first is the plural resource; the second is the quantity value, which is used to tell the application which grammatical form of the word to display; and the third value is defined only when the actual quantity is to be displayed to the user, substituting `%d` with the actual integer value.

When internationalizing your application, managing the translation of words properly and accounting for quantity in a particular language is very important. Not all languages follow the same rules for quantity, so in order to make this process manageable, using plural string resource files will definitely help.

For a particular word, you are able to define many different grammatical forms. To define more than one quantity form of a given word in your string resource file, you simply specify more than one `<item>` element and provide a value for the quantity attribute for that particular `<item>` denoting the word's quantity.

Working with String Arrays

You can specify lists of strings in resource files. This can be a good way to store menu options and drop-down list values. String arrays are defined in XML under the `res/values` project directory and compiled into the application package at build time.

String arrays are appropriately tagged with the `<string-array>` tag and a number of `<item>` child tags, one for each String in the array. Here is an example of a simple array resource file, `res/values/arrays.xml`:

```
<?xml version="1.0" encoding="utf-8"?>

<resources>

    <string-array name="flavors">

        <item>Vanilla</item>

        <item>Chocolate</item>

        <item>Strawberry</item>

        <item>Coffee</item>

        <item>Sherbet</item>

    </string-array>

    <string-array name="soups">

        <item>Vegetable minestrone</item>

        <item>New England clam chowder</item>

        <item>Organic chicken noodle</item>

    </string-array>

</resources>
```

As shown earlier in this chapter, accessing string-array resources is easy. The method `getStringArray()` retrieves a string array from a resource file, in this case, one named `flavors`:

```
String[] aFlavors =

    getResources().getStringArray(R.array.flavors);
```

Working with Boolean Resources

Other primitive types are supported by the Android resource hierarchy as well. Boolean resources can be used to store information about application game preferences and default values. Boolean resources are defined in XML under the `res/values` project directory and compiled into the application package at build time.

Defining Boolean Resources in XML

Boolean values are appropriately tagged with the <bool> tag and represent a name/value pair. The name attribute is how you refer to the specific Boolean value programmatically, so name these resources wisely.

Here is an example of the Boolean resource file res/values/bools.xml:

```
<?xml version="1.0" encoding="utf-8"?>

<resources>

    <bool name="onePlusOneEqualsTwo">true</bool>

    <bool name="isAdvancedFeaturesEnabled">false</bool>

</resources>
```

Using Boolean Resources Programmatically

To use a Boolean resource in code, you can load it using the getBoolean() method of the Resources class. The following code accesses your application's Boolean resource named bAdvancedFeaturesEnabled:

```
boolean isAdvancedMode =

    getResources().getBoolean(R.bool.isAdvancedFeaturesEnabled);
```

Working with Integer Resources

In addition to strings and Boolean values, you can also store integers as resources. Integer resources are defined in XML under the res/values project directory and compiled into the application package at build time.

Defining Integer Resources in XML

Integer values are appropriately tagged with the <integer> tag and represent a name-value pair. The name attribute is how you refer to the specific integer programmatically, so name these resources wisely.

Here is an example of the integer resource file res/values/nums.xml:

```
<?xml version="1.0" encoding="utf-8"?>

<resources>

    <integer name="numTimesToRepeat">25</integer>

    <integer name="startingAgeOfCharacter">3</integer>

</resources>
```

Using Integer Resources Programmatically

To use the integer resource, you must load it using the Resources class. The following code accesses your application's integer resource named `numTimesToRepeat`:

```
int repTimes = getResources().getInteger(R.integer.numTimesToRepeat);
```

TIP

Similar to string arrays, you can create integer arrays as resources using the `<integer-array>` tag with child `<item>` tags, defining one for each item in the array. You can then load the integer array using the `getIntArray()` method of the Resources class.

Working with Colors

Android applications can store RGB color values, which can then be applied to other screen elements. You can use these values to set the color of text or other elements, such as the screen background. Color resources are defined in XML under the `res/values` project directory and compiled into the application package at build time.

Defining Color Resources in XML

RGB color values always start with the hash symbol (`#`). The alpha value can be given for transparency control. The following color formats are supported:

- `#RGB` (for example, `#F00` is 12-bit color, red)
- `#ARGB` (for example, `#8F00` is 12-bit color, red with alpha 50%)
- `#RRGGBB` (for example, `#FF00FF` is 24-bit color, magenta)
- `#AARRGGBB` (for example, `#80FF00FF` is 24-bit color, magenta, with alpha 50%)

Color values are appropriately tagged with the `<color>` tag and represent a name/value pair. Here is an example of a simple color resource file, `res/values/colors.xml`:

```
<?xml version="1.0" encoding="utf-8"?>

<resources>

    <color name="background_color">#006400</color>

    <color name="text_color">#FFE4C4</color>

</resources>
```

Using Color Resources Programmatically

The example at the beginning of the chapter accessed a color resource. Color resources are simply integers. You should use the `ContextCompat` class to access colors as of Android Marshmallow 6.0 API Level 23, and pass in the application context. The following example shows the method `getColor()` retrieving a color resource called `prettyTextColor`:

```
int myResourceColor =

    ContextCompat.getColor(context, R.color.prettyTextColor);
```


Working with Dimensions

Many user interface layout controls, such as text controls and buttons, are drawn to specific dimensions. These dimensions can be stored as resources. Dimension values always end with a unit of measurement tag.

Defining Dimension Resources in XML

Dimension values are tagged with the <dimen> tag and represent a name/value pair. Dimension resources are defined in XML under the res/values project directory and compiled into the application package at build time.

Unit of Measurement	Description	Resource Tag Required	Example
Pixels	Actual screen pixels	px	20px
Inches	Physical measurement	in	1in
Millimeters	Physical measurement	mm	1mm
Points	Common font measurement unit	pt	14pt
Screen density-independent pixels	Pixels relative to 160dpi screen (preferable for dimension screen compatibility)	dp	1dp
Scale-independent pixels	Best for scalable font display	sp	14sp

Table 6.5 Dimension Unit Measurements Supported

The dimension units supported are shown in Table 6.5.

Here is an example of a simple dimension resource file called res/values/dimens.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <dimen name="FourteenPt">14pt</dimen>
    <dimen name="OneInch">1in</dimen>
    <dimen name="TenMillimeters">10mm</dimen>
    <dimen name="TenPixels">10px</dimen>
</resources>
```

NOTE

Generally, dp is used for layouts and graphics, whereas sp is used for text. A device's default settings will usually result in dp and sp being the same. However, because the user can control the size of text when it's in sp units, you would not use sp for text where the font layout size was important, such as with a title. Instead, it's good for content text where the user's settings might be important (such as a really large font for the vision impaired).

Using Dimension Resources Programmatically

Dimension resources are simply floating-point values. The `getDimension()` method retrieves a dimension resource called `textPointSize`:

```
float myDimension =  
    getResources().getDimension(R.dimen.textPointSize);
```

WARNING

Be cautious when choosing dimension units for your applications. If you are planning to target multiple devices, with different screen sizes and resolutions, you need to rely heavily on the more scalable dimension units, such as dp and sp, as opposed to pixels, points, inches, and millimeters.

Working with Simple Drawables

You can specify simple colored rectangles by using the drawable resource type, which can then be applied to other screen elements. These drawable resource types are defined in specific paint colors, much as the color resources are defined.

Defining Simple Drawable Resources in XML

Simple paintable drawable resources are defined in XML under the `res/values` project directory and compiled into the application package at build time. Paintable drawable resources use the `<drawable>` tag and represent a name/value pair. Here is an example of a simple drawable resource file called `res/values/drawables.xml`:

```
<?xml version="1.0" encoding="utf-8"?>  
  
<resources>  
  
    <drawable name="red_rect">#F00</drawable>  
  
</resources>
```

Although it might seem a tad confusing, you can also create XML files that describe other Drawable subclasses, such as `ShapeDrawable`. Drawable XML definition files are stored in the `res/drawable` directory within your project, along with image files. This is not the same as storing `<drawable>` resources, which are paintable drawables. `ShapeDrawable` resources are stored in the `res/values` directory, as explained previously.

Here is a simple `ShapeDrawable` described in the file `res/drawable/red_oval.xml`:

```
<?xml version="1.0" encoding="utf-8"?>

<shape xmlns:android="http://schemas.android.com/apk/res/android"

    android:shape="oval">

    <solid android:color="#f00"/>

</shape>
```

Of course, we don't need to specify the size because it will scale automatically to the layout it's placed in, much like any vector graphics format.

Using Simple Drawable Resources Programmatically

Drawable resources defined with <drawable> are simply rectangles of a given color, which is represented by the Drawable subclass ColorDrawable. As of Android Marshmallow 6.0 API Level 23, you should use the ContextCompat class to access drawable resources and pass in the application context as the first parameter. The following code retrieves a ColorDrawable resource called redDrawable:

```
ColorDrawable myDraw = (ColorDrawable) ContextCompat.
    getDrawable(context, R.drawable.redDrawable);
```

NOTE

To learn how to define XML resources for a particular type of drawable and to learn how to access the different types of drawable resources in your code, see the Android documentation at <http://d.android.com/guide/topics/resources/drawable-resource.html>.

TIP

Use the res/mipmap/ directory for placing your application launcher icons. In the past, app launcher icons were stored in the drawables directory, but the mipmap folder is now the best-practice location for these assets.

Working with Images

Applications often include visual elements such as icons and graphics. Android supports several image formats that can be directly included as resources for your application. These image formats are shown in Table 6.7.

Supported Image Format	Description	Required Extension
Portable Network Graphics (PNG)	Preferred format (lossless)	.png
Nine-Patch Stretchable Graphics	Preferred format (lossless)	.9.png
Joint Photographic Experts Group (JPEG)	Acceptable format (lossy)	.jpg, .jpeg
Graphics Interchange Format (GIF)	Discouraged format	.gif
WebP (WEBP)	Android 4.0+	.webp

Table 6.7 Image Formats Supported in Android

These image formats are all well supported by popular graphics editors such as Adobe Photoshop, GIMP, and Microsoft Paint. Adding image resources to your project is easy. Simply drag the image asset into the res/drawable resource directory hierarchy and it will automatically be included in the application package.

WARNING

All resource filenames must be lowercase and simple (letters, numbers, and underscores only). This rule applies to all files, including graphics.

Working with Nine-Patch Stretchable Graphics

Android device screens, be they smartphones, tablets, wearables, TVs, or auto displays, come in various dimensions. It can be handy to use stretchable graphics to allow a single graphic that can scale appropriately for different screen sizes and orientations or different lengths of text. This can save you or your designer a lot of time in creating graphics for many different screen sizes.

Android supports Nine-Patch Stretchable Graphics for this purpose. Nine-Patch graphics are simply PNG graphics that have patches, or areas of the image, defined to scale appropriately, instead of the entire image scaling as one unit. Often the center segment is transparent or has a solid color for a background because it's the stretched part. As such, a common use for Nine-Patch graphics is to create frames and borders. Little more than the corners are needed, so a very small graphics file can be used to frame any size image or View control.

Nine-Patch Stretchable Graphics can be created from PNG files using the draw9patch tool included with the tools directory of the Android SDK. We talk more about compatibility and using Nine-Patch graphics in Chapter 13, "Designing Compatible Applications."

Using Image Resources Programmatically

Image resources are simply another kind of Drawable called a BitmapDrawable. Most of the time, you need only the resource ID of the image to set as an attribute on a user interface control.

For example, if we drop the graphics file flag.png into the res/drawable directory and add an ImageView control to the main layout, we can interact with that control programmatically in the layout by first

using the `findViewById()` method to retrieve a control by its identifier and then casting it to the proper type of control—in this case, an `ImageView` (`android.widget.ImageView`) object:

```
ImageView flagImageView =
```

```
    (ImageView)findViewById(R.id.ImageView01);
```

```
flagImageView.setImageResource(R.drawable.flag);
```

Similarly, if you want to access the `BitmapDrawable` (`android.graphics.drawable.BitmapDrawable`) object directly, you can request that resource directly using the `getDrawable()` method, as follows:

```
BitmapDrawable bitmapFlag = (BitmapDrawable)
```

```
    ContextCompat.getDrawable(context, R.drawable.flag);
```

```
int iBitmapHeightInPixels =
```

```
    bitmapFlag.getIntrinsicHeight();
```

```
int iBitmapWidthInPixels = bitmapFlag.getIntrinsicWidth();
```

Finally, if you work with Nine-Patch graphics, the call to `getDrawable()` will return a `NinePatchDrawable` (`android.graphics.drawable.NinePatchDrawable`) object instead of a `BitmapDrawable` object:

```
NinePatchDrawable stretchy = (NinePatchDrawable)
```

```
    ContextCompat.getDrawable(context, R.drawable.pyramid);
```

```
int iStretchyHeightInPixels =
```

```
    stretchy.getIntrinsicHeight();
```

```
int iStretchyWidthInPixels = stretchy.getIntrinsicWidth();
```

Working with Animation

Android provides two categories of animations. The first category, property animation, allows you to animate an object's properties. The second category of animation is view animation. There are two different types of view animations: frame-by-frame animation and tween animations.

Frame-by-frame animation involves the display of a sequence of images in rapid succession. Tweened animation involves applying standard graphical transformations such as rotations and fades to a single image.

The Android SDK provides some helper utilities for loading and using animation resources. These utilities are found in the `android.view.animation.AnimationUtils` class. Let's look at how you define the different view animations in terms of resources.

Defining and Using Frame-by-Frame Animation Resources

Frame-by-frame animation is often used when the content changes from frame to frame. This type of animation can be used for complex frame transitions—much like a kid’s flip book.

To define frame-by-frame resources, take the following steps:

1. Save each frame graphic as an individual drawable resource. It may help to name your graphics sequentially, in the order in which they are displayed—for example, frame1.png, frame2.png, and so on.
2. Define the animation set resource in an XML file within the res/drawable/ resource directory hierarchy.
3. Load, start, and stop the animation programmatically.

Here is an example of a simple frame-by-frame animation resource file called res/drawable/juggle.xml that defines a simple three-frame animation that takes 1.5 seconds to complete a single loop:

```
<?xml version="1.0" encoding="utf-8" ?>

<animation-list

  xmlns:android="http://schemas.android.com/apk/res/android"

  android:oneshot="false">

  <item

    android:drawable="@drawable/splash1"

    android:duration="500" />

  <item

    android:drawable="@drawable/splash2"

    android:duration="500"/>

  <item

    android:drawable="@drawable/splash3"

    android:duration="500"/>

</animation-list>
```

Frame-by-frame animation set resources defined with <animation-list> are represented by the Drawable subclass AnimationDrawable. The following code retrieves an AnimationDrawable resource called juggle:

```
AnimationDrawable jugglerAnimation = (AnimationDrawable) ContextCompat.

getDrawable(context, R.drawable.juggle);
```

After you have a valid AnimationDrawable (android.graphics.drawable.Animation Drawable), you can assign it to a View control on the screen and start and stop animation.

Defining and Using Tweened Animation Resources

Tweened animation features include scaling, fading, rotation, and translation. These actions can be applied simultaneously or sequentially and might use different interpolators.

Tweened animation sequences are not tied to a specific graphics file, so you can write one sequence and then use it for a variety of different graphics. For example, you can make moon, star, and diamond graphics all pulse using a single scaling sequence, or you can make them spin using a rotate sequence.

Graphics animation sequences can be stored as specially formatted XML files in the `res/anim` directory and are compiled into the application binary at build time.

Here is an example of a simple animation resource file called `res/anim/spin.xml` that defines a simple rotate operation—rotating the target graphic counterclockwise four times in place, taking 10 seconds to complete:

```
<?xml version="1.0" encoding="utf-8" ?>
<set xmlns:android="http://schemas.android.com/apk/res/android"
    android:shareInterpolator="false">
    <rotate
        android:fromDegrees="0"
        android:toDegrees="-1440"
        android:pivotX="50%"
        android:pivotY="50%"
        android:duration="10000" />
    </set>
```

Using Tweened Animation Sequence Resources Programmatically

If we go back to the earlier example of a `BitmapDrawable`, we can now include some animation simply by adding the following code to load the animation resource file `spin.xml` and set the animation in motion:

```
ImageView flagImageView =
    (ImageView)findViewById(R.id.ImageView01);
flagImageView.setImageResource(R.drawable.flag);
...
Animation an =
    AnimationUtils.loadAnimation(this, R.anim.spin);
```

```
flagImageView.startAnimation(an);
```

Now you have your graphic spinning. Notice that we loaded the animation using the base class object `Animation`. You can also extract specific animation types using the subclasses that match: `RotateAnimation`, `ScaleAnimation`, `TranslateAnimation`, and `AlphaAnimation` (found in the `android.view.animation` package). There are a number of different interpolators you can use with your tweened animation sequences.

Working with Menus

You can also include menu resources in your project files. Similar to animation resources, menu resources are not tied to a specific control but can be reused in any menu control.

Defining Menu Resources in XML

Each menu resource (which is a set of individual menu items) is stored as a specially formatted XML file in the `res/menu` directory and is compiled into the application package at build time.

Here is an example of a simple menu resource file called `res/menu/speed.xml` that defines a short menu with four items in a specific order:

```
<menu xmlns:android="http://schemas.android.com/apk/res/android">
```

```
<item
```

```
    android:id="@+id/start"
```

```
    android:title="Start!"
```

```
    android:orderInCategory="1"></item>
```

```
<item
```

```
    android:id="@+id/stop"
```

```
    android:title="Stop!"
```

```
    android:orderInCategory="4"></item>
```

```
<item
```

```
    android:id="@+id/accel"
```

```
    android:title="Vroom! Accelerate!"
```

```
    android:orderInCategory="2"></item>
```

```
<item
```

```
    android:id="@+id/decel"
```

```
    android:title="Decelerate!"
```



```
        android:orderInCategory="3"></item>
</menu>
```

You can create menus using Android Studio, which can access the various configuration attributes for each menu item. In the previous case, we set the title (label) of each menu item and the order in which the items display. Now, you can use string resources for those titles instead of typing in the strings. For example:

```
<menu xmlns:android=
    "http://schemas.android.com/apk/res/android">

    <item
        android:id="@+id/start"
        android:title="@string/start"
        android:orderInCategory="1"></item>

    <item
        android:id="@+id/stop"
        android:title="@string/stop"
        android:orderInCategory="2"></item>

</menu>
```

Using Menu Resources Programmatically

To access the preceding menu resource called `res/menu/speed.xml`, simply override the method `onOptionsItemSelected()` in your Activity class, returning `true` to cause the menu to be displayed:

```
public boolean onOptionsItemSelected(Menu menu) {
    getMenuInflater().inflate(R.menu.speed, menu);
    return true;
}
```

That's it. Now, if you run your application and press the Menu button, you see the menu. A number of other XML attributes can be assigned to menu items. For a complete list of these attributes, see the Android SDK Reference for menu resources at the website <http://d.android.com/guide/topics/resources/menu-resource.html>. You will learn a lot more about menus and menu event handling in Chapter 7, "Exploring Building Blocks."

Working with XML Files

You can include arbitrary XML resource files to your project. You should store these XML files in the `res/xml` directory, and they are compiled into the application package at build time.

The Android SDK has a variety of packages and classes available for XML manipulation. You will learn more about XML handling in Chapter 15, “Accessing Files and Directories.” For now, we create an XML resource file and access it through code.

Defining Raw XML Resources

First, put a simple XML file in the `res/xml` directory. In this case, the file `my_pets.xml` with the following contents can be created:

```
<?xml version="1.0" encoding="utf-8"?>

<pets>

  <pet name="Bit" type="Bunny" />

  <pet name="Nibble" type="Bunny" />

  <pet name="Stack" type="Bunny" />

  <pet name="Queue" type="Bunny" />

  <pet name="Heap" type="Bunny" />

  <pet name="Null" type="Bunny" />

  <pet name="Nigiri" type="Fish" />

  <pet name="Sashimi II" type="Fish" />

  <pet name="Kiwi" type="Lovebird" />

</pets>
```

Using XML Resources Programmatically

Now you can access this XML file as a resource programmatically in the following manner:

```
XmlResourceParser myPets =

    getResources().getXml(R.xml.my_pets);
```

You can then use the parser of your choice to parse the XML. We discuss working with files, including XML files, in Chapter 15, “Accessing Files and Directories.”

Working with Raw Files

Your application can also include raw files as part of its resources. For example, your application might use raw files such as audio files, video files, and other file formats not supported by the Android SDK.

TIP

For a full list of supported media formats, have a look at the following Android documentation:
<http://d.android.com/guide/appendix/media-formats.html>.

Defining Raw File Resources

All raw resource files are included in the res/raw directory and are added to your package without further processing.

WARNING

All resource filenames must be lowercase and simple (letters, numbers, and underscores only). This also applies to raw file filenames even though the tools do not process these files other than to include them in your application package.

The resource filename must be unique to the directory and should be descriptive because the filename (without the extension) becomes the name by which the resource is accessed.

Using Raw File Resources Programmatically

You can access raw file resources from the res/raw resource directory and any resource from the res/drawable directory (bitmap graphics files or anything not using the <resource> XML definition method). Here is one way to open a file called the_help.txt:

InputStream iFile =

```
getResources().openRawResource(R.raw.the_help);
```

References to Resources

You can reference resources instead of duplicating them. For example, your application might need to reference a single string resource in multiple string arrays.

The most common use of resource references is in layout XML files, where layouts can reference any number of resources to specify attributes for layout colors, dimensions, strings, and graphics. Another common use is within style and theme resources.

Resources are referenced using the following format:

@resource_type/variable_name

Recall that earlier we had a string array of soup names. If we want to localize the soup listing, a better way to create the array is to create individual string resources for each soup name and then store the references to those string resources in the string array (instead of the text).

To do this, we define the string resources in the res/strings.xml file like this:

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<resources>
```

```
    <string name="app_name">Application Name</string>
```

```
    <string name="chicken_soup">Organic chicken noodle</string>
```

```
<string name="minestrone_soup">Veggie minestrone</string>
<string name="chowder_soup">New England clam chowder</string>
</resources>
```

Then we can define a localizable string array that references the string resources by name in the res/arrays.xml file like this:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string-array name="soups">
        <item>@string/minestrone_soup</item>
        <item>@string/chowder_soup</item>
        <item>@string/chicken_soup</item>
    </string-array>
</resources>
```

TIP

Save the strings.xml file first so that the string resources (that are included in the R.java class) are defined prior to trying to save the arrays.xml file, which references those particular string resources. Otherwise, you might get the following error:

Error: No resource found that matches the given name.

You can also use references to make aliases to other resources. For example, you can alias the system resource for the OK String to an application resource name by including the following in your strings.xml resource file:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string id="app_ok">@android:string/ok</string>
</resources>
```

You learn more about all the different system resources available later in this chapter.

TIP

Much as with string and integer arrays, you can create arrays of any type of resource by using the `<array>` tag with child `<item>` tags, defining one item for each resource in the array. You can then load the array of miscellaneous resources using the `obtainTypedArray()` method of the `Resources` class. The typed array resource is commonly used for grouping and loading a bunch of drawable resources with a single call. For more information, see the Android SDK documentation on typed array resources.

Referencing System Resources

In addition to the resources included in your project, you can also take advantage of the generic resources provided as part of the Android SDK. You can access system resources much as you would your own resources. The android package contains all kinds of resources, which you can browse by looking in the `android.R` subclasses. Here, you find system resources for

- Animation sequences for fading in and out
- Arrays of email/phone types (home, work, and such)
- Standard system colors
- Dimensions for application thumbnails and icons
- Many commonly used drawable and layout types
- Error strings and standard button text
- System styles and themes

You can reference system resources in other resources such as layout files by specifying the `@android` package name before the resource. For example, to set the background to the system color for darker gray, you set the appropriate background color attribute to `@android:color/darker_gray`.

You can access system resources programmatically through the `android.R` class. If we go back to our animation example, we could have used a system animation instead of defining our own. Here is the same animation example again, except it uses a system animation to fade in:

```
ImageView flagImageView =  
    (ImageView)findViewById(R.id.ImageView01);  
flagImageView.setImageResource(R.drawable.flag);  
Animation an = AnimationUtils.  
    loadAnimation(this, android.R.anim.fade_in);  
flagImageView.startAnimation(an);
```

WARNING

Although referencing system resources can be useful to give your application a look that is more consistent with the rest of a particular device's user interface (something users will appreciate), you still need to be cautious when doing so. If a particular device has system resources that are dramatically different or it fails to include specific resources your application relies on, your application may not look right or behave as expected. An installable application, called `rs:ResEnum`

(<https://play.google.com/store/apps/details?id=com.risesoftware.rsresourceenumerator>), can be used to enumerate and display the various system resources available on a given device. Thus, you can quickly verify system resource availability across your target devices.

Summary

Android applications rely on various types of resources, including strings, string arrays, colors, dimensions, drawable objects, graphics, animation sequences, layouts, and more. Resources can also be raw files. Many of these resources are defined with XML and organized into specially named project directories. Both default and alternative resources can be defined using this resource hierarchy.

Resources are compiled and accessed using the `R.java` class file, which is automatically generated by the Android Studio when the application resources are saved, allowing developers to access the resources programmatically.

Quiz Questions

1. True or false: All graphics are stored under the `res/graphics` directory structure.
2. What are the various resource types supported by the Android SDK?
3. What Resources method would you use to retrieve a string resource?
4. What Resources method would you use to retrieve a string array resource?
5. What image formats does the Android SDK support?
6. What is the format for referencing resources?

Exercises

1. Using the Android documentation, create a list of the different types of drawable resources.
2. Using the Android documentation, create a list of the available keyword values for the `quantity` attribute of the `<item>` element for quantity strings (`<plurals>`).
3. Provide an example of `TypedArray` defined in XML.